

Υλοποιητική Άσκηση - Τεχνικές Εξόρυξης Δεδομένων

Χρήστος Μάριος Νικολόπουλος - 1100644

Εαρινό Εξάμηνο 2025-2026

Περιεχόμενα

1 Περιβάλλον Υλοποίησης & Εγκατάσταση	3
1.1 Γλώσσα και Αρχιτεκτονική Περιβάλλοντος	3
1.2 Αναλυτική Καταγραφή Βιβλιοθηκών Λογισμικού	3
1.3 Βήματα Εγκατάστασης & Εκτέλεσης	3
2 Διαδικασία Υλοποίησης (Architecture & Pipeline)	4
2.1 Στάδιο 1: Φόρτωση Δεδομένων (DataLoader)	4
2.2 Στάδιο 2: Προεπεξεργασία & Καθαρισμός (Preprocess)	4
2.3 Στάδιο 3: Επιβλεπόμενη Μάθηση (SupervisedLearning)	5
2.4 Στάδιο 4: Μη Επιβλεπόμενη Μάθηση (UnsupervisedLearning)	5
3 Σχολιασμός Τελικών Αποτελεσμάτων (Evaluation & Discussion)	7
3.1 Προετοιμασία Δεδομένων & Ανισορροπία Κλάσεων	7
3.2 Επιβλεπόμενη Μάθηση (Supervised Learning)	7
3.2.1 Logistic Regression (Λογιστική Παλινδρόμηση)	7
3.2.2 Decision Tree & Random Forest (Δενδρικά Μοντέλα)	7
3.3 Μη Επιβλεπόμενη Μάθηση (Unsupervised Learning & Clustering)	8
3.3.1 K-Means & Ιεραρχική Ομαδοποίηση (Agglomerative)	8
3.3.2 DBSCAN	8
3.4 Τελικό Συμπέρασμα Σύγκρισης	8

1 Περιβάλλον Υλοποίησης & Εγκατάσταση

1.1 Γλώσσα και Αρχιτεκτονική Περιβάλλοντος

Σύμφωνα με τις προδιαγραφές της εργαστηριακής άσκησης, ως γλώσσα υλοποίησης ορίστηκε η **Python** (έκδοση ≥ 3.10). Για την ανάπτυξη και τη διασφάλιση της σταθερότητας του κώδικα, δημιουργήθηκε ένα απομονωμένο εικονικό περιβάλλον (*virtual environment* - *.venv*). Αυτή η πρακτική αποτρέπει τις διενέξεις μεταξύ των εκδόσεων των βιβλιοθηκών του συστήματος και εξασφαλίζει την πλήρη αναπαραγωγικότητα των πειραμάτων.

1.2 Αναλυτική Καταγραφή Βιβλιοθηκών Λογισμικού

Για την κάλυψη των αναγκών των τριών ερωτημάτων (EDA, Classification, Clustering), επιλέχθηκαν και χρησιμοποιήθηκαν οι εξής εξειδικευμένες βιβλιοθήκες:

- **pandas & numpy**: Αποτελούν τον πυρήνα της διαχείρισης δεδομένων. Χρησιμοποιήθηκαν για τη φόρτωση των CSV αρχείων, τον εντοπισμό και την αφαίρεση διπλότυπων, τη διαχείριση ελλειπών τιμών, καθώς και για όλους τους απαραίτητους μετασχηματισμούς των πινάκων (DataFrames).
- **scikit-learn (sklearn)**: Η κεντρική βιβλιοθήκη μηχανικής μάθησης. Μέσω αυτής υλοποιήθηκαν οι τεχνικές κανονικοποίησης, οι αλγόριθμοι επιβλεπόμενης μάθησης (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier), η βελτιστοποίηση υπερπαραμέτρων (GridSearchCV, StratifiedKFold), καθώς και οι αλγόριθμοι ομαδοποίησης (KMeans, AgglomerativeCluster, DBSCAN).
- **matplotlib & seaborn**: Βιβλιοθήκες οπτικοποίησης. Χρησιμοποιήθηκαν για τη σχεδίαση των πινάκων συσχέτισης (Heatmaps) και των πινάκων σύγχυσης (Confusion Matrices), οι οποίοι αποθηκεύονται αυτόματα σε μορφή εικόνας (.png).

1.3 Βήματα Εγκατάστασης & Εκτέλεσης

Οι εξαρτήσεις του έργου έχουν καταγραφεί συγκεντρωτικά στο αρχείο `requirements.txt`. Η διαδικασία στησίματος του περιβάλλοντος από το τερματικό περιλαμβάνει τα εξής βήματα:

```
1 cd path/to/Data_mining_project/src
2
3 python -m venv .venv
4
5 .venv\Scripts\activate
6
7 pip install --upgrade pip
8 pip install -r requirements.txt
9
10 python src/pipeline.py
11
```

Listing 1: Εντολές Εγκατάστασης και Εκτέλεσης Pipeline

2 Διαδικασία Υλοποίησης (Architecture & Pipeline)

Η αρχιτεκτονική του συστήματος βασίζεται σε αντικειμενοστραφή σχεδιασμό (Object-Oriented Programming). Κάθε στάδιο του πειράματος έχει απομονωθεί σε ξεχωριστό αρχείο κώδικα (module), ενώ η συνολική ροή ενορχηστρώνεται από ένα κεντρικό pipeline αρχείο (pipeline.py).

2.1 Στάδιο 1: Φόρτωση Δεδομένων (DataLoader)

Η αρχική ανάγνωση των δεδομένων υλοποιείται μέσω της κλάσης DataLoader στο αρχείο data_loader.py. Η μέθοδος load_data() εντοπίζει δυναμικά τον κατάλογο εργασίας και χρησιμοποιεί τη βιβλιοθήκη glob για να συλλέξει όλα τα αρχεία μορφής .csv από το μονοπάτι ../data/raw. Για κάθε αρχείο, πραγματοποιείται αφαίρεση κενών διαστημάτων από τους τίτλους των στηλών (columns.str.strip()) για την αποφυγή σφαλμάτων αναφοράς, και όλα τα επιμέρους DataFrames ενοποιούνται με την pd.concat().

```
1 def load_data(self):
2     current_dir = os.getcwd()
3     path = os.path.join(current_dir, "..", "data", "raw")
4     all_files = glob.glob(os.path.join(path, "*.csv"))
5     print("Loading Data")
6     list_df = []
7     for f in all_files:
8         temp_df = pd.read_csv(f, low_memory=False)
9         temp_df.columns = temp_df.columns.str.strip()
10        list_df.append(temp_df)
11        self.df = pd.concat(list_df, ignore_index=True)
12    return self.df
13
```

Listing 2: Υλοποίηση Φόρτωσης Δεδομένων στην DataLoader

2.2 Στάδιο 2: Προεπεξεργασία & Καθαρισμός (Preprocess)

Η κλάση Preprocess (preprocess.py) αναλαμβάνει την εκκαθάριση του dataset πριν από τη μοντελοποίηση:

- **Διαχείριση Ελλειπών & Απειρών Τιμών:** Οι τιμές απείρου (inf, -inf) αντικαθίστανται με NaN. Οι στήλες Flow Bytes/s και Flow Packets/s συμπληρώνονται με τη μέση τιμή τους (fillna), ενώ τυχόν εναπομείναντα κενά διαγράφονται οριστικά (dropna).
- **Αφαίρεση Διπλοτύπων:** Η μέθοδος removing_dupl() απομακρύνει τις πανομοιότυπες εγγραφές, θωρακίζοντας τα μοντέλα από το φαινόμενο του overfitting.
- **Επιλογή Χαρακτηριστικών (Feature Selection):** Υπολογίζεται ο πίνακας συσχέτισης (corr()).abs(). Με κατώφλι threshold=0.95, η μέθοδος εντοπίζει και αφαιρεί τις στήλες του άνω τριγωνικού πίνακα που παρουσιάζουν υπερβολική πολυγραμμικότητα, μειώνοντας τη διαστασιμότητα.

```
1 def feature_selection(self, threshold=0.95):
2     numeric_df = self.df.select_dtypes(include=np.number)
3     corr_matrix = numeric_df.corr().abs()
4     upper = corr_matrix.where(
5         np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
6     )
7     to_drop = [col for col in upper.columns if any(upper[col] >
8         threshold)]
9     self.df.drop(columns=to_drop, inplace=True)
10    print(f"\nFeature selection (threshold={threshold}): removed {len(
11        to_drop)} columns")
```

```

10     return to_drop
11

```

Listing 3: Μέθοδος Feature Selection βάσει Συσχέτισης

2.3 Στάδιο 3: Επιβλεπόμενη Μάθηση (Supervised Learning)

Η κλάση Supervised Learning (modeling.py) διαχειρίζεται την εκπαίδευση των διεργασιών ταξινόμησης.

1. **Διαχωρισμός & Κανονικοποίηση:** Στο pipeline.py απομονώνεται στρωματοποιημένο δείγμα **50.000 εγγράφων**. Η prepare_data() χωρίζει το δείγμα σε Train (60%), Validation (20%) και Test (20%) με stratify=y, και ακολουθεί τυποποίηση των χαρακτηριστικών με StandardScaler.
2. **Στρατηγική Grid Search:** Για τα μοντέλα Logistic Regression, Decision Tree και Random Forest (με class_weight='balanced'), εκτελούνται δύο σενάρια: το Scenario 1 (χρήση PredefinedSplit για αξιολόγηση στο Validation set) και το Scenario 2 (κλασική διασταυρούμενη επικύρωση 5-fold CV στο Train set).

```

1     def run_classification(self, option):
2         classifier = self.select_classifier(option)
3         params      = self.get_grid(option)
4         model_name  = MODEL_NAMES[option]
5
6         # Scenario 1 - Predefined split (Train/Val Split)
7         split       = [-1] * len(self.X_train) + [0] * len(self.X_val)
8         pds         = PredefinedSplit(test_fold=split)
9         X_combined  = np.concatenate((self.X_train, self.X_val), axis=0)
10        y_combined  = np.concatenate((self.y_train, self.y_val), axis=0)
11
12        grid_split = GridSearchCV(classifier, params, cv=pds, scoring="
f1_weighted", n_jobs=-1)
13        grid_split.fit(X_combined, y_combined)
14        best_split = self.evaluate(grid_split, "Scenario 1 - Train/Val
Split", option)
15
16        # Scenario 2 - 5 fold cross validation
17        grid_cv = GridSearchCV(classifier, params, cv=5, scoring='
f1_weighted', n_jobs=-1)
18        grid_cv.fit(self.X_train, self.y_train)
19        best_cv = self.evaluate(grid_cv, "Scenario 2 - 5-fold CV", option)
20

```

Listing 4: Υλοποίηση των δύο Σεναρίων Grid Search

2.4 Στάδιο 4: Μη Επιβλεπόμενη Μάθηση (Unsupervised Learning)

Η κλάση Unsupervised Learning (clustering.py) εκτελείται σε υποσύνολο **5.000 δειγμάτων** χωρίς τη χρήση του Label. Πραγματοποιείται εύρεση βέλτιστου k (Elbow method & Silhouette analysis) για τον K-Means, εκτέλεση Ιεραρχικής Ομαδοποίησης με διάφορα κριτήρια σύνδεσης (ward, complete, average), και Grid Search για τον DBSCAN με μεταβολή των παραμέτρων eps και min_samples.

```

1     def dbscan_grid_search(self, eps_values=(0.3, 0.5, 1.0, 2.0),
2         min_samples_values=(5, 10, 20)):
3         results = []
4         for eps in eps_values:
5             for ms in min_samples_values:
6

```

```

5     model = DBSCAN(eps=eps, min_samples=ms, n_jobs=-1)
6     labels = model.fit_predict(self.X)
7     n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
8
9     if n_clusters >= 2:
10        sil = silhouette_score(self.X, labels, sample_size=10000,
random_state=42)
11    else:
12        sil = -1.0
13        results.append({'eps': eps, 'min_samples': ms, 'silhouette': sil})
14        best = max(results, key=lambda r: r['silhouette'])
15    return self.run_dbscan(eps=best['eps'], min_samples=best['
min_samples']), best
16

```

Listing 5: Διαδικασία DBSCAN Grid Search

3 Σχολιασμός Τελικών Αποτελεσμάτων (Evaluation & Discussion)

3.1 Προετοιμασία Δεδομένων & Ανισορροπία Κλάσεων

Το αρχικό σύνολο δεδομένων (CIC-IDS-2017) χαρακτηρίζεται από τεράστιο όγκο (2.830.743 εγγραφές και 79 χαρακτηριστικά). Η εφαρμογή του φιλτραρίσματος για την αφαίρεση άπειρων (*inf*) και ελλিপών (*NaN*) τιμών, σε συνδυασμό με τη μέθοδο επιλογής χαρακτηριστικών (Feature Selection) με κατώφλι συσχέτισης $\text{threshold} = 0.95$, οδήγησε στην αφαίρεση 23 περιττών στηλών, αφήνοντας 55 τελικά χαρακτηριστικά για τη μοντελοποίηση.

Το πιο κρίσιμο στοιχείο που καθορίζει τη συμπεριφορά όλων των μοντέλων είναι η **ακραία ανισορροπία κλάσεων (Extreme Class Imbalance)**. Η κλάση 0 (BENIGN) κυριαρχεί καθολικά με πάνω από 2 εκατομμύρια δείγματα, ενώ μειονοτικές κλάσεις όπως η 8 (Heartbleed) εμφανίζουν μόλις 11 δείγματα σε ολόκληρο το dataset. Για τον λόγο αυτό, η στρωματοποιημένη δειγματοληψία 50.000 γραμμών ήταν απαραίτητη για να καταστεί υπολογιστικά εφικτή η διαδικασία εκπαίδευσης.

3.2 Επιβλεπόμενη Μάθηση (Supervised Learning)

Η αξιολόγηση των τριών ταξινομητών πραγματοποιήθηκε σε test set 10.000 δειγμάτων, συγκρίνοντας τη συμπεριφορά τους στο Scenario 1 (Train/Val Split) και στο Scenario 2 (5-fold Cross Validation).

3.2.1 Logistic Regression (Λογιστική Παλινδρόμηση)

Η Λογιστική Παλινδρόμηση παρουσίασε τη χαμηλότερη συνολική απόδοση με συνολική ακρίβεια $\text{Accuracy} = 0.89$ και $\text{Macro Avg } F_1\text{-Score} = 0.53$. Λόγω της γραμμικής της φύσης, αδυνατεί να διαχωρίσει ικανοποιητικά τις μειονοτικές κλάσεις:

- Στην κλάση 1, εμφάνισε εξαιρετικά χαμηλό $\text{Precision} = 0.01$ αλλά υψηλό $\text{Recall} = 0.86$ (ή 0.71 στο CV). Αυτό υποδηλώνει ότι το μοντέλο υπερ-προβλέπει τη συγκεκριμένη επίθεση, παράγοντας τεράστιο αριθμό ψευδώς θετικών αποτελεσμάτων (False Positives).
- Στις πολύ σπάνιες κλάσεις (12, 14), το $F_1\text{-score}$ κυμάνθηκε κοντά στο μηδέν (0.03).
- Τα αποτελέσματα μεταξύ των δύο σεναρίων ήταν σχεδόν πανομοιότυπα με τις ίδιες βέλτιστες παραμέτρους ($C = 10$, $\text{max_iter} = 1000$), αποδεικνύοντας ότι το μοντέλο έχει σταθερή συμπεριφορά αλλά πάσχει από *underfitting*.

3.2.2 Decision Tree & Random Forest (Δενδρικά Μοντέλα)

Αντίθετα με τη γραμμική προσέγγιση, οι δενδρικοί αλγόριθμοι επέδειξαν **εξαιρετική συμπεριφορά** αγγίζοντας το $\text{Accuracy} = 1.00$ και $\text{Weighted Avg } F_1\text{-Score} = 1.00$.

- Το **Decision Tree** πέτυχε κορυφαίες επιδόσεις με τις μειονοτικές κλάσεις (π.χ. κλάση 1 με $F_1 = 0.80$ και κλάση 12 με $F_1 = 0.73$ στο Scenario 1). Η χρήση του κριτηρίου *entropy* με $\text{max_depth}=20$ προσέφερε την καλύτερη ισορροπία. Στο 5-fold CV, το μοντέλο χωρίς περιορισμό βάθους (None) διατήρησε την υψηλή του απόδοση ($\text{Macro } F_1 = 0.82$).
- Το **Random Forest** (με βέλτιστες παραμέτρους 200 δέντρα και $\text{max_depth} = 20$ στο CV) επιβεβαίωσε την ανωτερότητά του ως ensemble μέθοδος. Εμφάνισε εξαιρετικό $F_1\text{-score} \geq 0.92$ για σχεδόν όλες τις κλάσεις επιθέσεων.
- **Εξαίρεση στην κλάση 14:** Λόγω του ελάχιστου support (3 δείγματα), κανένας από τους αλγόριθμους δεν κατάφερε να την εντοπίσει σωστά ($F_1 = 0.00$), καθώς το μέγεθος του δείγματος δεν επαρκούσε για την εξαγωγή κανόνων.

Συμπέρασμα Ταξινόμησης: Τα δενδρικά μοντέλα είναι ιδανικά για την ανίχνευση δικτυακών εισβολών, καθώς οι κυβερνοεπιθέσεις συχνά αφήνουν διακριτά «αποτυπώματα» που εκφράζονται μέσω αυστηρών ιεραρχικών κατωφλίων (if-then rules) στα χαρακτηριστικά της ροής (π.χ. συγκεκριμένα Destination Ports ή Flow Durations).

3.3 Μη Επιβλεπόμενη Μάθηση (Unsupervised Learning & Clustering)

Η συσταδοποίηση πραγματοποιήθηκε σε 5.000 δείγματα χωρίς τη χρήση της μεταβλητής Label. Η σύγκριση των αποτελεσμάτων με τις πραγματικές ετικέτες αναδεικνύει τη γεωμετρική δομή του χώρου.

3.3.1 K-Means & Ιεραρχική Ομαδοποίηση (Agglomerative)

Η ανάλυση Silhouette υπέδειξε ως βέλτιστο αριθμό συστάδων το $k = 2$ με ένα εξαιρετικά υψηλό σκορ Silhouette = 0.9752 και πολύ χαμηλό δείκτη Davies-Bouldin (DB = 0.0168).

Εξετάζοντας τους πίνακες συσχέτισης των clusters με τα πραγματικά Labels (Confusion Matrices ομαδοποίησης), παρατηρούμε ότι τόσο ο **K-Means** ($k = 2$) όσο και όλες οι παραλλαγές της **Ιεραρχικής Ομαδοποίησης** (Ward, Complete, Average) παρήγαγαν **ακριβώς το ίδιο αποτέλεσμα**:

- Το **Cluster 0** απορρόφησε σχεδόν το σύνολο των δειγμάτων (4159 BENIGN και σχεδόν όλες τις επιθέσεις).
- Το **Cluster 1** περιέχει **μόνο 1 δείγμα** (το οποίο ανήκει στην κλάση 0).

Αυτή η συμπεριφορά, σε συνδυασμό με το γεγονός ότι η αύξηση των συστάδων σε $k = 3$ έριξε κατακόρυφα το Silhouette score στο 0.4414, υποδηλώνει την ύπαρξη ενός **ακραίου απομακρυσμένου σημείου (extreme outlier)** στα δεδομένα. Οι αλγόριθμοι που βασίζονται σε αποστάσεις (όπως ο K-Means και ο Hierarchical) αναγκάζονται να δημιουργήσουν ένα ξεχωριστό cluster μόνο για αυτό το outlier σημείο προκειμένου να ελαχιστοποιήσουν το σφάλμα, αποτυγχάνοντας να διαχωρίσουν τις πραγματικές επιθέσεις από τη φυσιολογική κίνηση. Επιπλέον, η PCA έδειξε ότι οι 2 πρώτες κύριες συνιστώσες ερμηνεύουν το 41.07% της συνολικής διασποράς, επιβεβαιώνοντας ότι ο χώρος έχει υψηλή πολυπλοκότητα.

3.3.2 DBSCAN

Ο DBSCAN, ως αλγόριθμος βασισμένος στη πυκνότητα, συμπεριφέρθηκε τελείως διαφορετικά. Η βέλτιστη παραμετροποίηση που προέκυψε από το Grid Search ήταν $\text{eps} = 2.0$ και $\text{min_samples} = 5$ (Silhouette = 0.3109).

Ο αλγόριθμος εντόπισε 28 διαφορετικές συστάδες και απομόνωσε 414 σημεία ως θόρυβο (Noise points). Ο έλεγχος του πίνακα συσχέτισης δείχνει ότι ο DBSCAN κατάφερε να «σπάσει» τη μάζα των δεδομένων σε πολλές μικρές, πυκνές υποομάδες:

- Η φυσιολογική κίνηση (κλάση 0) διασκορπίστηκε κυρίως στα Clusters 0, 1, 4, 5.
- Συγκεκριμένες επιθέσεις ομαδοποιήθηκαν μαζί με μεγάλη πυκνότητα. Για παράδειγμα, η κλάση 4 έχει ισχυρή παρουσία στο Cluster 1 (791 δείγματα) και στο Cluster 5 (1739 δείγματα).

3.4 Τελικό Συμπέρασμα Σύγκρισης

Η σύγκριση των δύο προσεγγίσεων δείχνει ότι για το συγκεκριμένο πρόβλημα ανίχνευσης κυβερνοεπιθέσεων, η **Επιβλεπόμενη Μάθηση (και συγκεκριμένα το Random Forest)** είναι σαφώς ανώτερη και έτοιμη για πρακτική εφαρμογή, καθώς μπορεί να διαχειριστεί την ανισορροπία των κλάσεων μέσω κατάλληλων βαρών ($\text{class_weight} = \text{'balanced'}$).

Αντίθετα, η **Μη Επιβλεπόμενη Μάθηση** επηρεάζεται έντονα από την παρουσία ακραίων τιμών (outliers) και την κυριαρχία της κλάσης BENIGN. Ωστόσο, ο DBSCAN αποδείχθηκε πολύ πιο χρήσιμος από τον K-Means, καθώς η ικανότητά του να αναγνωρίζει αυθαίρετα σχήματα πυκνότητας του επέτρεψε να διακρίνει υποομάδες επιθέσεων (όπως της κλάσης 4), καθιστώντας τον ένα καλό εργαλείο για την ανίχνευση νέων, άγνωστων μορφών ανωμαλιών στο δίκτυο.